

Lab-2

Exercise-2:

```
1 from logic import *
2
3 rain = Symbol("rain")
4 hagrid = Symbol("hagrid")
5 dumbledore = Symbol("dumbledore")
6 snape = Symbol("snape")
7
8 knowledge = And(
9     Implication(Not(rain), hagrid),
10    Or(hagrid, dumbledore, snape),
11    Not(And(hagrid, dumbledore)),
12    Not(And(hagrid, snape)),
13    Not(And(dumbledore, snape)),
14    Implication(rain, Not(snape)),
15    snape,
16    dumbledore
17 )
18
19 print(model_check(knowledge, rain))
20
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\ar7_1\Downloads\Sophomore-2\CS 3081\Labs\Lab 2> python -u "c:\Users\ar7_1\Downloads\Sophomore-2\CS 3081\Labs\Lab 2\harry.py"

True

PS C:\Users\ar7_1\Downloads\Sophomore-2\CS 3081\Labs\Lab 2>

Exercise-3:

```
harry.py clue.py
1 import termcolor
2 from logic import *
3
4 alice = Symbol("AliceSuspect")
5 bob = Symbol("BobSuspect")
6 carol = Symbol("CarolSuspect")
7 characters = [alice, bob, carol]
8
9 office = Symbol("office")
10 garage = Symbol("garage")
11 basement = Symbol("basement")
12 rooms = [office, garage, basement]
13
14 poison = Symbol("poison")
15 rope = Symbol("rope")
16 candlestick = Symbol("candlestick")
17 weapons = [poison, rope, candlestick]
18
19 symbols = characters + rooms + weapons
20
21 def check_knowledge(knowledge):
22     for symbol in symbols:
23         if model_check(knowledge, symbol):
24             termcolor.cprint(f"{symbol}: YES", "green")
25         elif not model_check(knowledge, Not(symbol)):
26             print(f"{symbol}: MAYBE")
27
28
29 knowledge = And(
30     Or(alice, bob, carol),
31     Or(office, garage, basement),
32     Or(poison, rope, candlestick)
33 )
34
35 knowledge.add(And(
36     Not(alice), Not(office), Not(poison)
37 ))
38
39 knowledge.add(Or(
40     Not(bob), Not(garage), Not(rope)
41 ))
42
43 knowledge.add(Not(candlestick))
44 knowledge.add(Not(basement))
45
46 check_knowledge(knowledge)
```

PROBLEMS OUTPUT TERMINAL

PS C:\Users\ar7_1\Downloads\Sophomore-2\CS 3081\Labs\Lab 2> python -u "c:\Users\ar7_1\Downloads\Sophomore-2\CS 3081\Labs\Lab 2\clue.py"

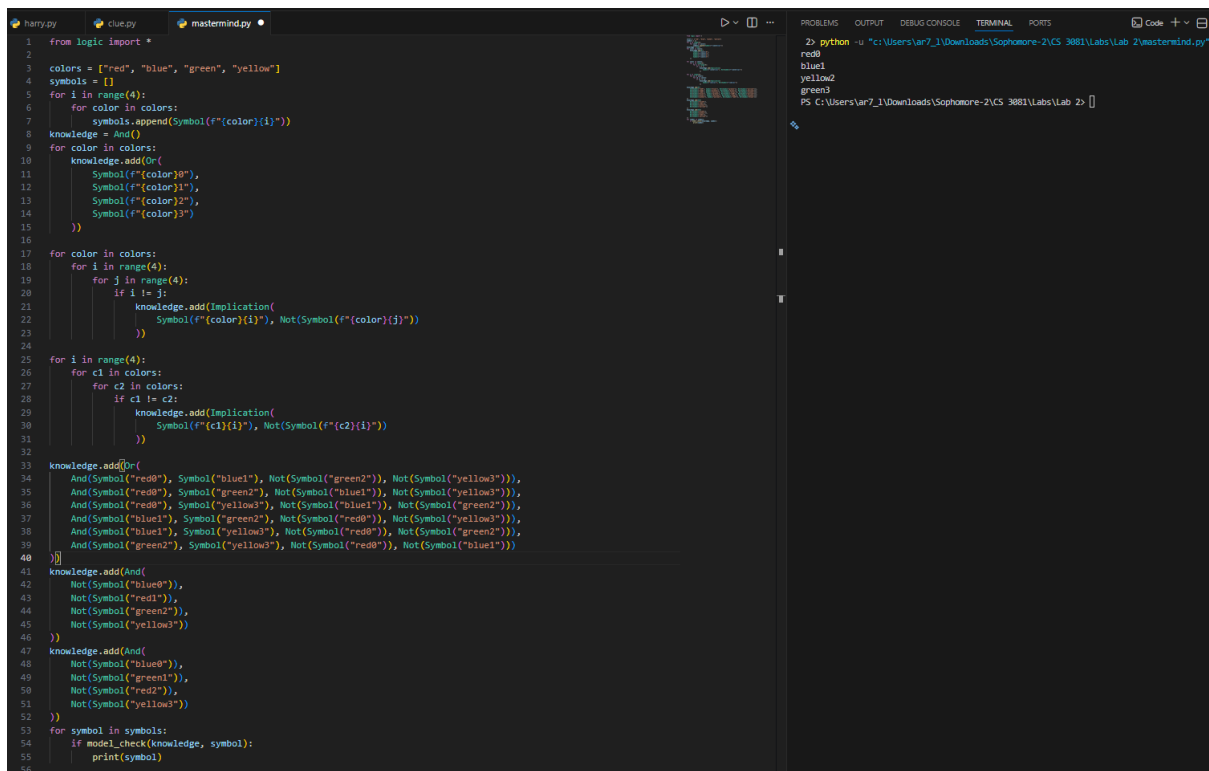
AliceSuspect: YES

garage: YES

rope: YES

PS C:\Users\ar7_1\Downloads\Sophomore-2\CS 3081\Labs\Lab 2>

Exercise-4:



```
1 from logic import *
2
3 colors = ["red", "blue", "green", "yellow"]
4 symbols = []
5 for i in range(4):
6     for color in colors:
7         symbols.append(Symbol(f"{color}{i}"))
8 knowledge = And()
9 for color in colors:
10     knowledge.add(Or(
11         Symbol(f"{color}0"),
12         Symbol(f"{color}1"),
13         Symbol(f"{color}2"),
14         Symbol(f"{color}3")
15     ))
16
17 for color in colors:
18     for i in range(4):
19         for j in range(4):
20             if i != j:
21                 knowledge.add(implication(
22                     Symbol(f"{color}{i}"), Not(Symbol(f"{color}{j}"))
23                 ))
24
25 for i in range(4):
26     for c1 in colors:
27         for c2 in colors:
28             if c1 != c2:
29                 knowledge.add(implication(
30                     Symbol(f"{c1}{i}"), Not(Symbol(f"{c2}{i}"))
31                 ))
32
33 knowledge.add(Or(
34     And(Symbol("red0"), Symbol("blue1"), Not(Symbol("green2")), Not(Symbol("yellow3"))),
35     And(Symbol("red0"), Symbol("green2"), Not(Symbol("blue1")), Not(Symbol("yellow3"))),
36     And(Symbol("red0"), Symbol("yellow3"), Not(Symbol("blue1")), Not(Symbol("green2"))),
37     And(Symbol("blue1"), Symbol("green2"), Not(Symbol("red0")), Not(Symbol("yellow3"))),
38     And(Symbol("blue1"), Symbol("yellow3"), Not(Symbol("red0")), Not(Symbol("green2"))),
39     And(Symbol("green2"), Symbol("yellow3"), Not(Symbol("red0")), Not(Symbol("blue1")))
40 ))
41 knowledge.add(And(
42     Not(Symbol("blue0")),
43     Not(Symbol("red1")),
44     Not(Symbol("green2")),
45     Not(Symbol("yellow3"))
46 ))
47 knowledge.add(And(
48     Not(Symbol("blue0")),
49     Not(Symbol("green1")),
50     Not(Symbol("red2")),
51     Not(Symbol("yellow3"))
52 ))
53 for symbol in symbols:
54     if model_check(knowledge, symbol):
55         print(symbol)
```

To fully solve it, one would need additional clues that further restrict the remaining positions. Ones that give "1 correct position" feedback to pinpoint exact placements.

Exercise-5:

1. How many symbols does puzzle.py use? How many truth-table rows does model explore? (Formula: 2^n where n = number of symbols)

People: 4, Houses: 4 $\rightarrow$ symbols = $4 \times 4 = 16$ symbols

Truth-table rows = $2^{16} = 65,536$ rows

2. How many symbols does mastermind.py use? How many rows? Check.

Same as puzzle.py coincidentally.

3. If we extended mastermind.py to 6 colors and 6 positions, how many symbols would there be? How many rows?

Symbols = $6 \times 6 = 36$ symbols

Truth-table rows = $2^{36} = 68,719,476,736$ rows (~68 billion)

4. What algorithmic approach could replace truth-table enumeration for large problems? Name one and briefly explain how it improves on the exponential complexity

Instead of enumerating all 2^n rows, a SAT solver uses techniques like:

Unit propagation: if a clause has one unset literal, it must be true; this forces assignments early

Backtracking with pruning: explores only branches that haven't already been contradicted, cutting off huge portions of the search space